Finite Element Surface Processes Model

(Finite Element Method - Semi Lagrangian Method)

Reza Masoumzadeh

1 Surface Processes Model

Surface process models (SPMs) simulate the temporal evolution of topographic elevation, h, by applying material redistribution laws that govern erosion, sediment transport, and deposition. Different SPM formulations exist because the surface process considered to dominate landscape evolution varies among regions. In continental settings, fluvial processes and bedrock landsliding are generally regarded as the primary geomorphic controls, although glacial erosion and chemical weathering can also be significant in certain environments.

The evolution of the surface h is defined via the following advection-diffusion equation:

$$\begin{aligned} &\frac{\partial h}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H h = u_V + \nabla_H \cdot (\kappa(\mathbf{x}) \nabla_H h), \quad \text{in } \Omega, \\ &\frac{\partial h}{\partial \boldsymbol{n}} = 0, \qquad\qquad\qquad\qquad\qquad\qquad \text{on } \partial\Omega, \end{aligned} \tag{1}$$

where $\boldsymbol{u}$ is the velocity, sub-script H denotes quantities (or operators) defined in the horizontal plane, u_V denotes the component of the velocity in the vertical direction, $\kappa(\mathbf{x})$ represents a diffusivity, and $\boldsymbol{n}$ is the outward-pointing unit normal vector.

To solve Eq. (1), we adopt a first-order operator splitting technique in which the advective and diffusive and source terms are updated sequentially. The operator splitting technique updates the height h in the following manner: Solve the advection equation

$$\frac{\partial \widehat{h}}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H \widehat{h} = 0, \qquad t \in [t^k, t^k + \Delta t] \tag{2}$$

for the advected height $\widehat{h}$ using the following initial data $\widehat{h}(\mathbf{x}, t^k) = h(\mathbf{x}, t^k)$. Then solve

the diffusion equation

$$\frac{\partial h}{\partial t} = u_V(\mathbf{x}, t) + \nabla_H \cdot (\kappa(h, \mathbf{x}, t)\nabla_H h), \qquad t \in [t^k, t^k + \Delta t] \tag{3}$$

for $h(\mathbf{x}, t)$ using initial data $h(\mathbf{x}, t^k) = \widehat{h}(\mathbf{x}, t^k + \Delta t)$.

Forward Euler

We adopt an explicit (in time) finite difference approximation to solve Eq. (3), which given some initial height $h^k(\mathbf{x})$ defined at time $t = t^k$ yields

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V(\mathbf{x}, t^k) + \nabla_H \cdot (\kappa(h^k, \mathbf{x}, t^k)\nabla_H h^k), \tag{4}$$

where Δt is the time-step and h^{k+1} is the approximate value of $h(t^k + \Delta t)$. Note the non-linearity is evaluated using values from the previous time step k. The hyperbolic equation defined by Eq. (2) is solved using a semi-Lagrangian technique.

2 Finite Element Method for Solving the Diffusion Equation

We consider the following time-discrete surface evolution equation:

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V(\mathbf{x}, t^k) + \nabla_H \cdot \left(\kappa(h^k, \mathbf{x}, t^k)\nabla_H h^k\right). \tag{5}$$

Here, h denotes the surface elevation, u_V is the vertical velocity, κ is the surface diffusivity, and ∇_H denotes the horizontal gradient operator.

2.1 Constant Diffusivity κ

For the first FESPM implementation, we consider a constant diffusivity,

$$\kappa(h^k, \mathbf{x}, t^k) = \kappa, \tag{6}$$

so that Eq. (5) becomes

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V^k + \kappa\nabla_H^2 h^k. \tag{7}$$

Multiplying Eq. (7) by a test function v, and integrating over the computational domain Ω, gives

$$\int_\Omega \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega = \int_\Omega u_V^k v \, d\Omega + \int_\Omega \kappa\nabla_H^2 h^k v \, d\Omega. \tag{8}$$

Using integration by parts for the diffusion term,

$$\int_\Omega \kappa\nabla_H^2 h^k v \, d\Omega = -\int_\Omega \kappa\nabla_H h^k \cdot \nabla_H v \, d\Omega + \int_\Gamma (\kappa\nabla_H h^k \cdot \mathbf{n}) v \, d\Gamma. \tag{9}$$

For homogeneous Neumann boundary conditions,

$$\frac{\partial h}{\partial n} = 0 \qquad \text{on } \Gamma, \tag{10}$$

the boundary integral vanishes,

$$\int_{\Gamma} (\kappa \nabla_H h^k \cdot \mathbf{n}) v \, d\Gamma = 0. \tag{11}$$

Therefore, the weak formulation becomes

$$\int_{\Omega} \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega + \int_{\Omega} \kappa \nabla_H h^k \cdot \nabla_H v \, d\Omega = \int_{\Omega} u_V^k v \, d\Omega. \tag{12}$$

• If we consider a Neumann boundary condition as:

$$\kappa \frac{\partial h}{\partial n} = q_N \qquad \text{on } \Gamma_N, \tag{13}$$

therefore, the weak form is

$$\boxed{\int_{\Omega} \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega + \int_{\Omega} \kappa \nabla_H h^k \cdot \nabla_H v \, d\Omega = \int_{\Omega} u_V^k v \, d\Omega + \int_{\Gamma} q_N \, v \, d\Gamma} \tag{14}$$

Finite Element Approximation

The surface elevation is approximated using finite element basis functions,

$$h^k(\mathbf{x}) \approx \sum_{j=1}^{N} H_j^k \phi_j(\mathbf{x}), \tag{15}$$

and similarly,

$$h^{k+1}(\mathbf{x}) \approx \sum_{j=1}^{N} H_j^{k+1} \phi_j(\mathbf{x}), \tag{16}$$

where N is the total number of FEM nodes, ϕ_j are the finite element basis functions, and H_j^k represents the numerical value of h at node j at time level k.

Using the Galerkin method, the test function is chosen as

$$v = \phi_i. \tag{17}$$

Substituting Eqs. (45)–(17) into the weak formulation gives

$$\int_\Omega \frac{\sum_{j=1}^{N} H_j^{k+1}\phi_j - \sum_{j=1}^{N} H_j^k \phi_j}{\Delta t}\phi_i \, d\Omega + \int_\Omega \kappa \left(\sum_{j=1}^{N} H_j^k \nabla_H \phi_j \right) \cdot \nabla_H \phi_i \, d\Omega = \int_\Omega u_V^k \phi_i \, d\Omega. \quad (18)$$

Because the summations and integrals are linear,

$$\frac{1}{\Delta t}\sum_{j=1}^{N} \left(H_j^{k+1} - H_j^k\right) \int_\Omega \phi_j \phi_i \, d\Omega + \sum_{j=1}^{N} H_j^k \int_\Omega \kappa \nabla_H \phi_j \cdot \nabla_H \phi_i \, d\Omega = \int_\Omega u_V^k \phi_i \, d\Omega. \quad (19)$$

Mass Matrix

The FEM mass matrix is defined as

$$M_{ij} = \int_\Omega \phi_i \phi_j \, d\Omega. \quad (20)$$

Therefore,

$$\frac{1}{\Delta t}\sum_{j=1}^{N} M_{ij}\left(H_j^{k+1} - H_j^k\right) = \frac{1}{\Delta t}\left[M\left(\boldsymbol{H}^{k+1} - \boldsymbol{H}^k\right)\right]_i. \quad (21)$$

Diffusion Stiffness Matrix

The diffusion stiffness matrix is defined as

$$K_{ij} = \int_\Omega \kappa \nabla_H \phi_j \cdot \nabla_H \phi_i \, d\Omega. \quad (22)$$

Consequently,

$$\sum_{j=1}^{N} K_{ij} H_j^k = \left[K\boldsymbol{H}^k\right]_i. \quad (23)$$

Vertical Velocity Load Vector

The vertical velocity produces the load vector

$$F_i^k = \int_\Omega u_V^k \phi_i \, d\Omega. \tag{24}$$

Therefore,

$$\mathbf{F}^k = \left[F_1^k, F_2^k, \ldots, F_N^k\right]^T. \tag{25}$$

Derivation of $\mathbf{F}^k = \mathbf{M}\mathbf{u}_V^k$

When the source term u_V^k is defined by discrete nodal values rather than a continuous analytical function, it is interpolated using the same shape functions $\phi_j(\mathbf{x})$:

$$u_V^k(\mathbf{x}) \approx \sum_{j=1}^{N} (u_V^k)_j \phi_j(\mathbf{x}) \tag{26}$$

Substituting this interpolation into the definition of the load vector component F_i^k:

$$\begin{aligned} F_i^k &= \int_\Omega \phi_i(\mathbf{x}) u_V^k(\mathbf{x}) \, d\Omega & (27)\\ &= \int_\Omega \phi_i(\mathbf{x}) \left(\sum_{j=1}^{N} (u_V^k)_j \phi_j(\mathbf{x}) \right) d\Omega & (28)\\ &= \sum_{j=1}^{N} \left(\int_\Omega \phi_i(\mathbf{x}) \phi_j(\mathbf{x}) \, d\Omega \right) (u_V^k)_j & (29) \end{aligned}$$

Since the mass matrix is defined as $M_{ij} = \int_\Omega \phi_i(\mathbf{x}) \phi_j(\mathbf{x}) \, d\Omega$, the expression simplifies to:

$$F_i^k = \sum_{j=1}^{N} M_{ij} (u_V^k)_j \implies \mathbf{F}^k = \mathbf{M}\mathbf{u}_V^k \tag{30}$$

Benefit: This avoids recalculating spatial integrals at each time step, replacing numerical quadrature with a fast matrix-vector multiplication ($\mathbf{M}\mathbf{u}_V^k$).

Final FEM System

Combining the mass matrix, stiffness matrix, and vertical velocity load vector gives

$$\frac{1}{\Delta t}\mathbf{M}\left(\boldsymbol{H}^{k+1}-\boldsymbol{H}^{k}\right)+\mathbf{K}\boldsymbol{H}^{k}=\mathbf{F}^{k}. \tag{31}$$

Multiplying by Δt,

$$\mathbf{M}\left(\boldsymbol{H}^{k+1}-\boldsymbol{H}^{k}\right)=\Delta t\left(\mathbf{F}^{k}-\mathbf{K}\boldsymbol{H}^{k}\right). \tag{32}$$

Thus,

$$\mathbf{M}\boldsymbol{H}^{k+1}=\mathbf{M}\boldsymbol{H}^{k}+\Delta t\left(\mathbf{F}^{k}-\mathbf{K}\boldsymbol{H}^{k}\right). \tag{33}$$

Stability Analysis

1. Explicit Scheme

The explicit time integration yields the update formulation:

$$\boldsymbol{H}^{k+1} = \left(\mathbf{I} - \Delta t \mathbf{M}^{-1}\mathbf{K}\right) \boldsymbol{H}^{k} + \Delta t \mathbf{M}^{-1}\mathbf{F}^{k} \tag{34}$$

Linear stability requires the spectral radius of the amplification matrix $\mathbf{G}_{\text{exp}} = \mathbf{I} - \Delta t \mathbf{M}^{-1}\mathbf{K}$ to be bounded by unity ($|\lambda(\mathbf{G}_{\text{exp}})| \leq 1$):

$$|1 - \Delta t \lambda_i(\mathbf{M}^{-1}\mathbf{K})| \leq 1 \implies \Delta t \leq \frac{2}{\lambda_{\max}(\mathbf{M}^{-1}\mathbf{K})} \tag{35}$$

Note: The explicit method is *conditionally stable*. Because $\lambda_{\max} \propto \Delta x^{-2}$, spatial mesh refinement severely restricts the permissible time step ($\Delta t \propto \Delta x^2$).

2.2 Non-Linear Variable Diffusivity κ

The diffusivity κ is non-linear and spatially variable as it depends on the local surface slope magnitude $\|\nabla_H h\|$:

$$\kappa(\mathbf{x}) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1} \tag{36}$$

Parameters:

- κ_0: Base constant diffusivity coefficient.
- S_c: Critical threshold slope ($S_c = \tan\theta$).
- n_s: Non-linear exponent (commonly $n_s = 2$).
- $\|\nabla_H h\| = \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial z}\right)^2}$: Gradient magnitude.

FEM Impact: Because $\kappa = \kappa(h)$, the stiffness matrix $\mathbf{K}(h)$ depends on the state variable h, making the PDE non-linear. Consequently, κ must be re-evaluated at each Gauss quadrature point during numerical integration.

Weak Form Derivation for Nonlinear Diffusion

1. Strong Form

The governing diffusion equation is:

$$\frac{\partial h}{\partial t} = u_V + \nabla_H \cdot (\kappa \nabla_H h) \tag{37}$$

where ∇_H is the horizontal gradient operator.

2. Weighted Residual Form

We multiply by a test function v and integrate over the domain Ω:

$$\int_\Omega v \frac{\partial h}{\partial t}\, d\Omega = \int_\Omega v u_V\, d\Omega + \int_\Omega v \Big[\nabla_H \cdot (\kappa \nabla_H h)\Big]\, d\Omega \tag{38}$$

3. Integration by Parts & Divergence Theorem

Using the product rule, the diffusion term is expanded:

$$\int_\Omega v\Big[\nabla_H \cdot (\kappa \nabla_H h)\Big]\, d\Omega = \int_\Omega \nabla_H \cdot (v\kappa \nabla_H h)\, d\Omega - \int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega \tag{39}$$

Applying the Divergence Theorem to the first term on the right-hand side:

$$\int_\Omega \nabla_H \cdot (v\kappa \nabla_H h)\, d\Omega = \oint_{\partial\Omega} v\kappa \nabla_H h \cdot \mathbf{n}\, dS \tag{40}$$

Assuming no-flux boundary conditions (the boundary integral vanishes), we obtain the weak form:

$$\int_\Omega v \frac{\partial h}{\partial t}\, d\Omega + \int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega = \int_\Omega v u_V\, d\Omega \tag{41}$$

4. Constant vs. Non-Constant κ

If κ is a constant, it can be factored out of the integral:

$$\int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega = \kappa \int_\Omega \nabla_H v \cdot \nabla_H h\, d\Omega \tag{42}$$

However, if κ is a function, it **must** remain inside the integral because it varies spatially over the domain Ω.

In this model, κ is highly nonlinear and depends on the local gradient of h:

$$\kappa(\mathbf{x}) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1} \tag{43}$$

where the gradient magnitude is:

$$\|\nabla_H h\| = \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial z}\right)^2} \tag{44}$$

Because κ depends on $\nabla_H h$, it must be evaluated at every Gauss quadrature point during the numerical integration of the element matrices.

5. FEM approximation on P1 triangles

On the structured grid, each quadrilateral cell is split into two P_1 triangles. On each triangle T_e with vertices $\{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$ the field h is approximated by

$$h^k(\mathbf{x}) \;\approx\; \sum_{m=1}^{N} H_m^k \, \phi_m(\mathbf{x}), \tag{45}$$

where ϕ_m are the global linear (barycentric) shape functions, H_m^k are the global nodal values, and N is the total number of nodes. The summation index is called m to distinguish it from the local indices $i, j \in \{1, 2, 3\}$ used below.

When restricted to a single triangle T_e, only three of the shape functions in (45) are non-zero. Relabeling them by a local index $i \in \{1, 2, 3\}$:

$$h^k(\mathbf{x})\big|_{T_e} \;=\; H_1^k \, \phi_1 + H_2^k \, \phi_2 + H_3^k \, \phi_3, \tag{46}$$

where ϕ_i denotes the linear shape function associated with the i-th local vertex of T_e. Because the ϕ_i are linear, their gradients — and therefore $\nabla_H h$ — are constant on each element.

6. Gradient of the shape functions

Let A_e be the (signed) area of the triangle and let (i, m, n) be a cyclic permutation of $(1, 2, 3)$. Define the geometric coefficients

$$b_i = z_m - z_n, \qquad c_i = x_n - x_m, \tag{47}$$

so that, for $i = 1, 2, 3$,

$$\begin{aligned} b_1 &= z_2 - z_3, & c_1 &= x_3 - x_2, \\ b_2 &= z_3 - z_1, & c_2 &= x_1 - x_3, \\ b_3 &= z_1 - z_2, & c_3 &= x_2 - x_1. \end{aligned} \tag{48}$$

The linear shape function associated with vertex i is

$$\phi_i(x,z) = \frac{1}{2A_e}\Big[\, b_i\,(x - x_m) + c_i\,(z - z_m)\,\Big]. \tag{49}$$

This form satisfies, by construction,

$$\phi_i(x_m, z_m) = \phi_i(x_n, z_n) = 0, \qquad \phi_i(x_i, z_i) = 1. \tag{50}$$

Equivalently, (49) can be written as $\phi_i(x,z) = \big(a_i + b_i x + c_i z\big)/(2A_e)$, with $a_i = x_m z_n - x_n z_m$; the constant $a_i/(2A_e)$ fixes the normalisation $\phi_i(x_i, z_i) = 1$.

Differentiating (49),

$$\nabla_H \phi_i \;=\; \begin{pmatrix} \partial_x \phi_i \\ \partial_z \phi_i \end{pmatrix} \;=\; \frac{1}{2A_e} \begin{pmatrix} b_i \\ c_i \end{pmatrix}, \qquad i = 1,2,3, \tag{51}$$

which is a *constant* vector on T_e.

7. Gradient of h on a triangle

Substituting (46) into $\nabla_H h$ and using (51):

$$\nabla_H h\big|_{T_e} \;=\; \sum_{i=1}^{3} H_i^k\, \nabla_H \phi_i \;=\; \frac{1}{2A_e} \sum_{i=1}^{3} H_i^k \begin{pmatrix} b_i \\ c_i \end{pmatrix}. \tag{52}$$

Equivalently, in components:

$$\left(\frac{\partial h}{\partial x}\right)_e = \frac{1}{2A_e}\big(b_1 H_1^k + b_2 H_2^k + b_3 H_3^k\big), \tag{53}$$

$$\left(\frac{\partial h}{\partial z}\right)_e = \frac{1}{2A_e}\big(c_1 H_1^k + c_2 H_2^k + c_3 H_3^k\big). \tag{54}$$

Because b_i and c_i depend only on the triangle geometry, the gradient (52) is evaluated **once per triangle** from the three nodal values H_1^k, H_2^k, H_3^k — no quadrature is required.

8. Magnitude of the gradient

The scalar quantity that appears in the critical-slope law is

$$\|\nabla_H h\|_e \;=\; \sqrt{\left(\frac{\partial h}{\partial x}\right)_e^2 + \left(\frac{\partial h}{\partial z}\right)_e^2}. \tag{55}$$

Using (53)–(54):

$$\|\nabla_H h\|_e \;=\; \frac{1}{2A_e}\sqrt{\Big(\sum_{i=1}^{3} b_i H_i^k\Big)^2 + \Big(\sum_{i=1}^{3} c_i H_i^k\Big)^2}. \tag{56}$$

9. Element diffusivity κ_e

Since $\|\nabla_H h\|_e$ is constant on T_e, the diffusivity κ is also constant on that element; we denote it by κ_e. Using the critical-slope law

$$\kappa(\mathbf{x}, h) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1}, \tag{57}$$

we obtain the *element scalar*

$$\boxed{\kappa_e \;=\; \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|_e}{S_c}\right)^{n_s}\right]^{-1}} \tag{58}$$

with $\|\nabla_H h\|_e$ given by (56). In practice the ratio $\|\nabla_H h\|_e / S_c$ is capped below 1 (e.g. at $1 - 10^{-5}$) to avoid the singularity as the slope approaches the critical value.

When κ_0, S_c or n_s vary spatially, their values on element T_e are taken as the arithmetic mean over the three vertices:

$$\kappa_0 \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} \kappa_{0,i}, \qquad S_c \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} S_{c,i}, \qquad n_s \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} n_{s,i}. \tag{59}$$

10. Local stiffness matrix

The local stiffness matrix on T_e is

$$K_{ij}^{(e)} = \int_{T_e} \kappa \, \nabla_H \phi_i \cdot \nabla_H \phi_j \, d\Omega, \qquad i, j \in \{1, 2, 3\}. \tag{60}$$

Since both $\kappa = \kappa_e$ and $\nabla_H \phi_i \cdot \nabla_H \phi_j$ are constant on T_e, the integral reduces to a single multiplication by the area:

$$\boxed{K_{ij}^{(e)} = \frac{\kappa_e}{4\,A_e}\big(b_i b_j + c_i c_j\big)} \qquad i, j = 1, 2, 3. \tag{61}$$

This is the exact value of the integral for P_1 triangles — no numerical quadrature is required.

11. Assembly and time dependence

The global stiffness matrix is assembled by summing the local matrices over all triangles sharing each pair of nodes,

$$K_{MN} = \sum_{e} \sum_{\substack{i,j: \\ \mathrm{glob}(i)=M,\, \mathrm{glob}(j)=N}} K_{ij}^{(e)}, \tag{62}$$

where M, N denote global node numbers and $\mathrm{glob}(\cdot)$ maps the local index to the global index.

Because κ_e in (58) depends on $\|\nabla_H h\|_e$, which in turn depends on the nodal values H_i^k, the matrix K is *nonlinear* in h. The temporal discretisation therefore treats the diffusion operator explicitly:

$$\boldsymbol{M}\,\boldsymbol{H}^{n+1} = \boldsymbol{M}\,\boldsymbol{H}^{*} + \Delta t \left(\boldsymbol{M}\,\boldsymbol{U}_V - \boldsymbol{K}\big(\boldsymbol{H}^{*}\big)\,\boldsymbol{H}^{*}\right), \tag{63}$$

where $\boldsymbol{H}^{*}$ is the post-advection surface. The sequence at each diffusion step is:

1. For every triangle T_e, compute b_i and c_i from (47).

2. Compute $\nabla_H h|_{T_e}$ from (53)–(54).

3. Compute $\|\nabla_H h\|_e$ from (56).

4. Compute κ_e from (58).

5. Build the local matrix $K_{ij}^{(e)}$ from (61).

6. Assemble $K_{ij}^{(e)}$ into the global matrix K_{MN}.

7. Solve the linear system (63) with the consistent mass matrix $\boldsymbol{M}$.

Steps 2–6 are repeated at **every** diffusion step, because h, and therefore κ_e, changes with time. The cost is $O(N_{\text{elem}})$, negligible compared with the CG solve of (63).

3 Semi-Lagrangian Method for Solving the Advection Equation

3.1 Governing equation and basic idea

The horizontal transport of the surface elevation is described by the advection equation

$$\frac{\partial \widehat{h}}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H \widehat{h} = 0, \qquad \boldsymbol{x} \in \Omega, \quad t \in [t^n, t^{n+1}], \tag{64}$$

where $\widehat{h}(\boldsymbol{x}, t)$ denotes the surface elevation during the advection stage, $\boldsymbol{u}_H = (u_x, u_y)$ is the horizontal velocity vector, $\boldsymbol{x} = (x, y)$ is the horizontal position, and Ω is the computational domain. The horizontal gradient operator is defined as

$$\nabla_H = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right). \tag{65}$$

The time interval is divided into steps of length Δt, such that $t^{n+1} = t^n + \Delta t$. At the beginning of each advection step, the initial condition is

$$\widehat{h}(\boldsymbol{x}, t^n) = h^n(\boldsymbol{x}), \tag{66}$$

where h^n is the known surface-elevation field at time t^n.

The main idea of the semi-Lagrangian (SL) method is to determine where the material arriving at a grid point at the new time level came from. Instead of calculating the transported elevation only from information at neighboring grid points, the method follows the motion backward in time. It identifies the departure point of each grid point and then interpolates the known elevation field at that location.

For example, consider a grid point $\boldsymbol{x}_i$ at time t^{n+1}. During the time interval Δt, the material reaching this point may have originated at another location, $\boldsymbol{X}_{d,i}$, at time t^n. The elevation at the arrival point is therefore obtained from the elevation at this departure point. This procedure is repeated for every grid point in the computational domain.

In the coupled geodynamic model considered here, the horizontal velocity field is supplied by I3ELVIS. During each surface advection update, this velocity field is treated as fixed. Thus, the SL method transports the surface elevation using the velocity field provided for the current coupling step.

3.1.1 Characteristic curves

The mathematical basis of the SL method is the characteristic formulation of the advection equation. Consider a moving material point whose position is denoted by $\boldsymbol{X}(t)$. Its motion is described by

$$\frac{d\boldsymbol{X}}{dt} = \boldsymbol{u}_H(\boldsymbol{X}, t). \tag{67}$$

This equation states that the material point moves with the local horizontal velocity.

The change in surface elevation along this moving trajectory follows from the chain rule:

$$\frac{d}{dt}\widehat{h}(\boldsymbol{X}(t), t) = \frac{\partial \widehat{h}}{\partial t} + \frac{d\boldsymbol{X}}{dt} \cdot \nabla_H \widehat{h}. \tag{68}$$

Substituting Equation (67) into Equation (68) gives

$$\frac{d}{dt}\widehat{h}(\boldsymbol{X}(t), t) = \frac{\partial \widehat{h}}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H \widehat{h}. \tag{69}$$

According to the advection equation (Equation (64)), the right-hand side is zero. Consequently,

$$\frac{d}{dt}\widehat{h}(\boldsymbol{X}(t), t) = 0. \tag{70}$$

This result means that the surface elevation remains constant along a characteristic trajectory during the pure-advection stage. The elevation changes at a fixed grid point because material is transported into or out of that location, rather than because the elevation changes along the trajectory itself.

3.1.2 Departure and arrival points

The SL method uses the characteristic relation in the backward direction. Let $\boldsymbol{x}_i$ be a grid point at the new time t^{n+1}. This point is called the arrival point. The corresponding departure point, denoted by $\boldsymbol{X}_{d,i}$, is the location reached by tracing the trajectory backward from $\boldsymbol{x}_i$ to the previous time t^n.

The trajectory therefore satisfies

$$\boldsymbol{X}(t^{n+1}) = \boldsymbol{x}_i, \qquad \boldsymbol{X}(t^n) = \boldsymbol{X}_{d,i}. \tag{71}$$

Since the elevation is constant along a characteristic, the exact characteristic relation is

$$\widehat{h}(\boldsymbol{x}_i, t^{n+1}) = h^n(\boldsymbol{X}_{d,i}). \tag{72}$$

The departure point is generally not located at a grid node. For this reason, the value $h^n(\boldsymbol{X}_{d,i})$ is not usually available directly in the stored grid data. It must be estimated by interpolation from the known values of h^n.

The numerical calculation at each grid point therefore consists of two main operations:

1. Calculate the departure point by tracing the characteristic backward over the time interval Δt.
2. Interpolate the surface-elevation field at the departure point to obtain the value at the arrival point.

The accuracy of the final result depends on both operations. An accurate trajectory estimate is important for locating the departure point, while the interpolation method determines how the elevation field is reconstructed between grid points.

3.1.3 Backward integration using the Runge–Kutta method

The departure point is calculated by numerically integrating the characteristic equation backward in time. To express the backward trajectory as an initial-value problem, introduce a parameter $\tau \in [0, \Delta t]$ and define $\boldsymbol{Y}(\tau)$ such that

$$\frac{d\boldsymbol{Y}}{d\tau} = -\boldsymbol{u}_H(\boldsymbol{Y}), \qquad \boldsymbol{Y}(0) = \boldsymbol{x}_i. \tag{73}$$

Here, the velocity is treated as time-independent during the current advection step. The minus sign indicates that the trajectory is followed backward in time. After integrating over the full time step, the departure point is

$$\boldsymbol{X}_{d,i} = \boldsymbol{Y}(\Delta t). \tag{74}$$

The implementation first attempts to calculate the departure point using the fourth-order Runge–Kutta method (RK4). RK4 estimates the trajectory by evaluating the velocity at the beginning, at two intermediate locations, and at the end of the step. These evaluations are combined to obtain a more accurate estimate than a single velocity evaluation.

Define

$$\boldsymbol{g}(\boldsymbol{Y}) = -\boldsymbol{u}_H(\boldsymbol{Y}). \tag{75}$$

For a backward step of length Δt, the RK4 stages are

$$\boldsymbol{k}_1 = \boldsymbol{g}(\boldsymbol{Y}_0), \tag{76}$$

$$\boldsymbol{k}_2 = \boldsymbol{g}\left(\boldsymbol{Y}_0 + \frac{\Delta t}{2}\boldsymbol{k}_1\right), \tag{77}$$

$$\boldsymbol{k}_3 = \boldsymbol{g}\left(\boldsymbol{Y}_0 + \frac{\Delta t}{2}\boldsymbol{k}_2\right), \tag{78}$$

$$\boldsymbol{k}_4 = \boldsymbol{g}\left(\boldsymbol{Y}_0 + \Delta t\, \boldsymbol{k}_3\right), \tag{79}$$

where $\boldsymbol{Y}_0 = \boldsymbol{x}_i$. The departure-point estimate is then calculated as

$$\boldsymbol{X}_{d,i} = \boldsymbol{Y}_0 + \frac{\Delta t}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right). \tag{80}$$

The intermediate stages allow the method to account for changes in velocity with position. Even when the velocity field is fixed in time, its spatial variation can make the trajectory curved rather than straight.

The RK4 procedure requires the velocity to be evaluated at its intermediate trajectory locations. Near the boundary, one of these locations may lie outside the computational domain. The implementation checks the intermediate trajectory locations and, if an RK4 stage leaves the domain, abandons the RK4 calculation for that trajectory and uses a first-order Runge–Kutta method (RK1) instead.

3.1.4 First-order fallback and boundary intersection

The RK1 method uses a single velocity evaluation at the arrival point to estimate the backward trajectory. Its unconstrained departure-point estimate is

$$\boldsymbol{X}^*_{d,i} = \boldsymbol{x}_i - \Delta t\, \boldsymbol{u}_H(\boldsymbol{x}_i). \tag{81}$$

This estimate assumes that the velocity evaluated at the arrival point is representative of the trajectory over the time step.

After calculating the RK1 estimate, the implementation checks whether the departure point lies inside the computational domain. If the estimate is outside the domain, the code determines the intersection of the backward trajectory with the domain boundary and uses this intersection as the departure point. In this way, the departure point used by the interpolation procedure is kept within the computational domain.

The purpose of this procedure is to avoid evaluating the interpolation at an exterior

location for which the numerical surface-elevation field is not defined. The RK1 fallback is a practical boundary-handling procedure in the implementation; it should not be interpreted as an additional physical boundary condition for the advection equation.

3.1.5 Interpolation at the Departure Point

Once the departure point is determined, the advected elevation is obtained by interpolating the elevation field at the previous time step:

$$\widehat{h}_i^{n+1} = \mathcal{I}_{\mathrm{SL}}\left[h^n\right](\boldsymbol{X}_{d,i}), \tag{82}$$

where $\mathcal{I}_{\mathrm{SL}}$ denotes the interpolation operator and $\boldsymbol{X}_{d,i}$ is the departure point associated with the arrival node $\boldsymbol{x}_i$.

The implementation uses an upwind-oriented monotone cubic-spline interpolation. It reconstructs the elevation between grid nodes to estimate its value at the departure point. The cubic reconstruction captures smooth spatial variations, while the monotonicity treatment helps reduce nonphysical oscillations near sharp gradients.

Repeating this process for all grid nodes yields the elevation field after the advection step.

3.1.6 Boundary treatment in the implementation

The treatment of the computational boundary in the SL code involves two separate operations. The first occurs during departure-point calculation, whereas the second modifies the boundary-node elevations after the advection update.

Departure-point restriction. During backward trajectory integration, the code checks whether trajectory points leave the computational domain. If an intermediate RK4 point is outside the domain, the RK1 fallback is used. If the RK1 departure estimate is outside the domain, the code calculates the intersection of the backward trajectory with the boundary and uses that point for interpolation. Thus, the interpolation point is restricted to the computational domain.

This procedure specifies how the code handles a trajectory that reaches the boundary. It does not, by itself, prescribe an independent inflow value for the elevation field. It also

does not imply that the normal derivative of the elevation is zero at the boundary.

Explicit resetting of boundary-node elevations. After the SL advection update, the surface-evolution routine explicitly resets the elevation at boundary nodes. For example, at the left boundary, the code sets the boundary elevation equal to the adjacent interior-node elevation minus a fixed offset:

$$h_{0,j}^{n+1} = h_{1,j}^{n+1} - \varepsilon_b, \qquad \varepsilon_b = 0.01. \tag{83}$$

Here, the first index identifies the grid direction normal to the left boundary, and the second index identifies the position along that boundary. Similar updates are applied at the right, lower, and upper boundaries, using the corresponding adjacent interior node.

This reset is performed after the advection calculation and is separate from both the trajectory integration and the interpolation step. The fixed offset is part of the implemented model behavior. Its physical interpretation and units should be specified when the method is documented.

In particular, this boundary treatment is not equivalent to a homogeneous Neumann condition,

$$\frac{\partial h}{\partial n} = 0, \tag{84}$$

which would express a zero normal gradient. The nonzero offset in Equation (83) generally produces a nonzero difference between adjacent boundary and interior values. Therefore, the implemented boundary reset should be described as an explicit boundary-node adjustment rather than as a zero-Neumann condition.

3.1.7 Complete algorithm for one advection step

The main operations performed during one SL advection step are summarized below.

1. **Initialize the advection step.** Store the surface elevation h^n and use the horizontal velocity field supplied by the geodynamic model. The velocity is held fixed during this advection update.
2. **Select an arrival node.** Consider a grid point $\boldsymbol{x}_i$ at the new time level t^{n+1}.
3. **Calculate the departure point.** Trace the characteristic backward from $\boldsymbol{x}_i$ using the RK4 method.

4. **Check the trajectory.** If an intermediate RK4 trajectory point leaves the computational domain, abandon the RK4 calculation for this trajectory and calculate the departure point using RK1.

5. **Handle an exterior RK1 estimate.** If the RK1 departure estimate lies outside the domain, determine the intersection of the backward trajectory with the domain boundary and use that intersection as the departure point.

6. **Interpolate the elevation.** Evaluate the old elevation field at the departure point using the selected upwind monotone cubic-spline interpolation option:

$$\widehat{h}_i^{n+1} = \mathcal{I}_{\mathrm{SL}}[h^n](\boldsymbol{X}_{d,i}).$$

7. **Repeat for all grid nodes.** Perform the departure-point calculation and interpolation for each surface grid node to construct the advected field.

8. **Reset the boundary nodes.** Apply the explicit boundary-node adjustment in the surface evolution routine, using the adjacent interior elevation minus the fixed offset $\varepsilon_b = 0.01$.

9. **Continue the surface update.** Pass the resulting field to the next stage of the surface-evolution procedure, such as the diffusion stage when operator splitting is used.

3.1.8 Summary

The semi-Lagrangian method transports the surface elevation by tracing characteristics backward in time and interpolating the previous solution at the corresponding departure points. In the present implementation, departure points are calculated primarily using RK4, with an RK1 fallback when an intermediate RK4 trajectory point leaves the computational domain. If the RK1 estimate lies outside the domain, the departure point is restricted to the boundary before interpolation. The interpolation is performed using the selected upwind monotone cubic-spline option.

After the advection update, boundary-node elevations are explicitly adjusted using neighboring interior values and a fixed offset. This last operation is distinct from the characteristic calculation and does not impose a homogeneous Neumann condition. These details together define the implemented SL advection procedure used in the surface-evolution model.

4 Comparison with the Finite-Difference Formulation Used in FDSPM

The original FDSPM code discretizes the diffusion operator using a node-centered finite-difference method (FDM) on a uniform Cartesian grid. For constant diffusivity, a standard five-point Laplacian stencil is employed, whereas variable diffusivity is treated using a conservative flux formulation with face-based coefficient averaging. Time integration is performed using the forward Euler method.

In contrast, the present method employs a P_1 finite-element formulation on a triangular mesh. The diffusion operator is discretized through its weak formulation, and the stiffness matrix is assembled using piecewise linear basis functions. For spatially variable diffusivity, the diffusion coefficient can be evaluated elementwise and incorporated into the stiffness matrix. This formulation provides flexibility in the choice of mesh and allows the method to accommodate irregular domain geometries and nonuniform spatial resolution.

An important advantage of P_1 FEM is its ability to represent the solution using piecewise linear functions over triangular elements. Unlike standard finite differences on uniform Cartesian grids, FEM can be applied to unstructured meshes and supports local mesh refinement, allowing computational resources to be concentrated in regions with steep surface gradients or rapidly varying topography. Furthermore, the weak formulation provides a systematic framework for incorporating boundary conditions and spatially variable material properties.

For sufficiently smooth solutions and suitable mesh regularity, the standard error estimates for P_1 FEM are

$$\|h_{\text{exact}} - h\|_{L^2(\Omega)} = O(\ell^2), \qquad \|h_{\text{exact}} - h\|_{H^1(\Omega)} = O(\ell), \tag{85}$$

where h_{exact} is the exact solution, h is the numerical solution, and ℓ denotes the characteristic mesh size. These estimates describe the expected convergence behavior under the required regularity assumptions; the actual performance of both methods must be assessed through numerical convergence tests.

When both methods use forward Euler time integration, the explicit diffusion stability restriction typically scales as

$$\Delta t = O(\ell^2). \tag{86}$$

However, the precise stability limit depends on the discrete spatial operator, the mesh, the diffusivity, and, for FEM, the treatment of the mass matrix. Consequently, the time-step restrictions of the two methods should be evaluated separately.

FDM has the advantage of a relatively simple implementation and low computational cost on uniform Cartesian grids. In comparison, P_1 FEM offers greater geometric flexibility, supports unstructured triangular meshes and local refinement, and provides a natural framework for weak formulations and variable coefficients. These properties make FEM particularly suitable for problems involving irregular geometries or spatially varying surface properties.

Nevertheless, FEM is not universally more accurate or computationally efficient than FDM. A fair comparison requires equivalent physical parameters, boundary conditions, time-integration schemes, and reference solutions. Spatial and temporal convergence, computational cost, and solution accuracy should therefore be evaluated under consistent test conditions.